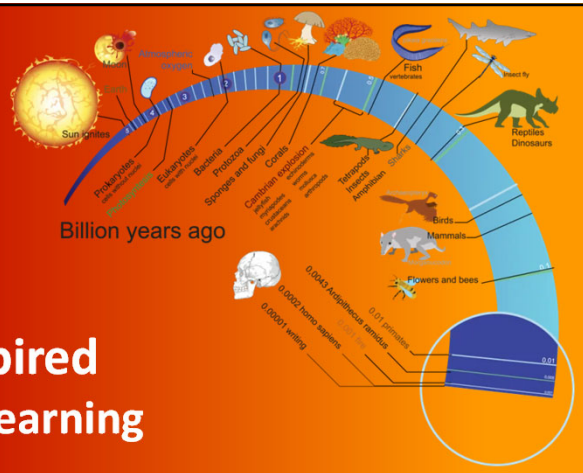
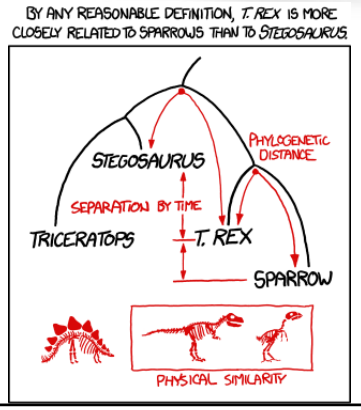
Bio-Inspired Machine Learning



1

Rationale



2

Artificial Intelligence And Machine Learning

What Is The Difference Between Artificial Intelligence And Machine Learning?

Artificial Intelligence is the broader concept of machines being able to carry out tasks in a way that we would consider "smart".

Vague

And,

Machine Learning is a current application of AI based around the idea that we should really just be able to give machines access to data and let them learn for themselves.

3

Artificial Intelligence And Machine Learning

Artificial Intelligence

- A broad field focused on creating machines that can perform tasks requiring *human-like intelligence*.
- Includes reasoning, decision-making, perception, language understanding, and problem-solving.
- AI systems may use rules, logic, knowledge, or learning from data.

Machine Learning (ML)

- A subset of AI that enables machines to *learn patterns from data*.
- Instead of explicitly programming every rule, ML algorithms learn from examples.
- Common applications include classification, prediction, detection, and recommendation.

4

Machine Learning

Machine learning is the science of getting computers to act without being explicitly programmed.

General

In the past decade, machine learning has given us self-driving cars, practical speech recognition, effective web search, and a vastly improved understanding of the human genome. Machine learning is so pervasive today that you probably use it dozens of times a day without knowing it.

5

Machine Learning

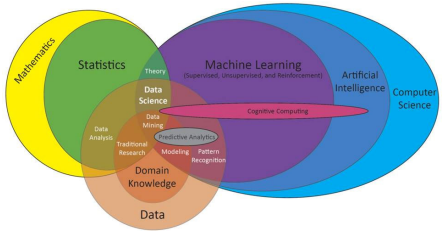
Machine learning is a method of data analysis that automates analytical model building.

It is a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

6

Machine Learning

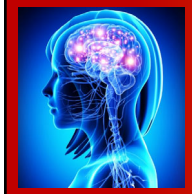
Machine learning is a method of data analysis that automates analytical model building.



It is a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

7

How to create AI?



Biological Intelligence

to

Machine Intelligence

8

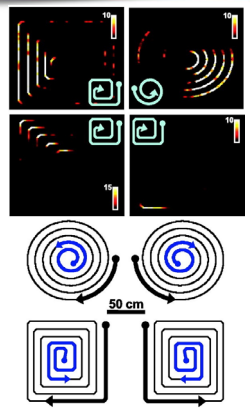
How to create AI?

Inspirations

- Principles of natural intelligence
- Evolution hypothesized by Charles Darwin
- Building blocks of knowledge

Thought Provoking

- Evolution has evolved neural structures that aid learning
- Can we identify those structures and recreate them in AI methods?



9

What Is Bio-Inspired Machine Learning?

Bio-Inspired Machine Learning (BIML) draws inspiration from biological systems, processes, and natural intelligence to develop machine learning models.

It treats *nature as a source of computational principles* for learning, adaptation, search, and decision-making.

BIML is inspired by how natural systems solve complex problems:

- *Brains and neurons* → neural networks and deep learning
- *Evolution and natural selection* → genetic algorithms and evolutionary computation
- *Social organisms* such as ants, bees, birds, and fish → swarm intelligence
- *Immune systems* → artificial immune systems and anomaly detection
- *Ecological* adaptation → adaptive and self-organising systems

10

10

Bio-Inspired Machine Learning

Bio-inspired ML does not copy nature exactly. It abstracts useful mechanisms from nature to design systems that can learn, adapt, optimise, and make decisions.

Bio-inspired methods are important because many real-world AI and data engineering problems are too large, noisy, dynamic, or poorly structured for conventional optimisation methods.

BIML aims to develop ML systems that are:

- *Adaptive*: able to respond to changing data, environments, and objectives
- *Robust*: able to tolerate noise, uncertainty, missing values, and incomplete information
- *Efficient*: able to search large solution spaces without exhaustive evaluation
- *Flexible*: suitable for discrete, continuous, mixed, and black-box optimisation problems
- *Explainable*: may be capable of producing interpretable rules or solution structures

11

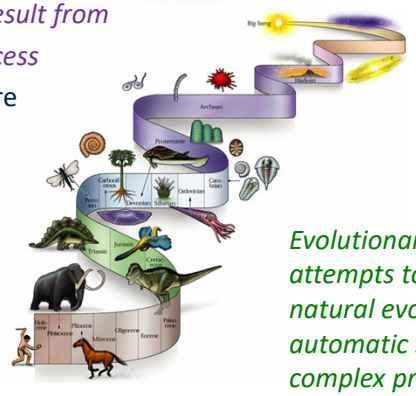
11

Evolutionary Inspiration

Biological systems result from an evolutionary process

Biological systems are

- robust
- complex
- adaptive



Evolutionary Computation attempts to copy process of natural evolution for automatic solution of complex problems

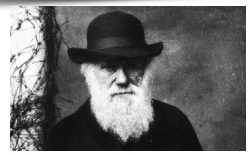
12

12

The 4 Pillars of Evolution

All species derive from common ancestor

Charles Darwin, 1859



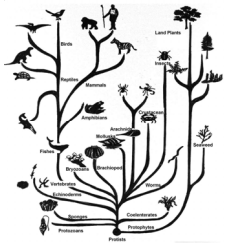
Population: Group of several individuals

Diversity: Individuals have different characteristics

Heredity: Characteristics are transmitted over generations

Selection:

- *Individuals make more offspring than the environment can support*
- *Better at food gathering = better at surviving = make more offspring*



13

13

Evolution without Progress

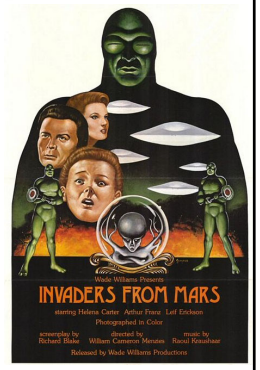
"why we should not fear an invasion from Mars"

(Gould, 1997)

Humans are not the top of the evolutionary ladder *(misleading image of evolution with humans at top or end).*

Evolution without Progress:

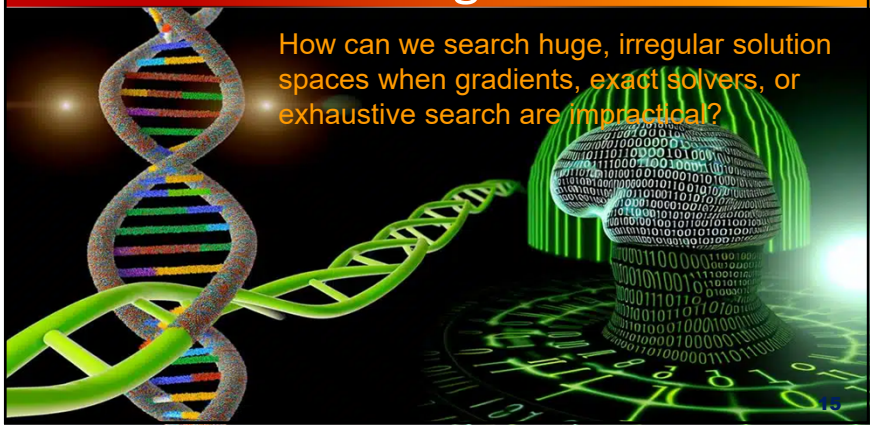
- If no competition, no selection of the fittest
- Individuals selected against current environment
- Accumulation of change with no cost or benefit (also known as Neutral Evolution)



14

14

Genetic Algorithms



15

15

Genetic Algorithms

- A type of evolutionary computation inspired by *Darwin's theory of evolution*.
- Developed by *John Holland*, University of Michigan (1970's)
  - To understand the adaptive processes of natural systems
  - To design artificial systems software that retains the robustness of natural systems

16

16

Genetic Algorithms

- A genetic algorithm generates a *population* of possible solutions encoded as chromosomes, evaluates their *fitness*, and create a new population by applying genetic *operators-crossover* and *mutation*.
- By repeating this process over many *generations*, the genetic algorithm breeds an optimal solution to the problem.
- Provide efficient, effective techniques for optimization and *machine learning* applications

17

17

Genetic Algorithms

- **Gene:** A *basic unit of chromosome* that controls the development of a particular feature of a living organism. A gene is generally represented by either 0 or 1.
- **Chromosomes:** A *string of genes* that represent an individual. Each chromosome consists of a number of gene.
- **Fitness:** The ability of a living organism to *survive and reproduce* in a specific environment.
- **Crossover:** A *reproduction operator* that creates a new chromosome by exchanging parts of two existing chromosome.

18

18

Genetic Algorithms

- **Mutation:** A *genetic operator* that randomly change the gene value in a chromosome.
- **Fitness Function:** A *mathematical function* used for calculating the fitness of a chromosome.
- **Offspring:** An individual that was produced through *reproduction*. It also referred to as *child*.
- **Population:** A group of individuals that *breed together*. It is a set of candidate solutions

19

19

Genetic Algorithm: How Does It Work?

1. Define the Problem
   - Specify the objective, constraints, and solution requirements.
2. Encode Solutions
   - Represent a candidate solution as a chromosome.
   - Decide how genes represent the solution.
3. Initialize Population
   - Generate an initial population of candidate solutions.
   - Solutions may be generated randomly or using prior knowledge.

20

20

Genetic Algorithm: How Does It Work?

4. Evaluate Fitness
   - Use a fitness function to measure how well each chromosome solves the problem.
   - Higher fitness → better solution.
5. Select Parents
   - Select chromosomes for reproduction.
   - Fitter individuals have a higher probability of being selected.
6. Apply Crossover
   - Combine genetic material from two parents.
   - Produces new offspring with characteristics from both parents.

21

21

Genetic Algorithm: How Does It Work?

7. Apply Mutation
   - Randomly change selected genes.
   - Maintains population diversity and explores new solutions.
8. Repair or Penalize
   - Repair offspring that violate constraints, when possible.
   - Alternatively, penalize invalid solutions through the fitness function.
9. Evaluate the Offspring
   - Calculate the fitness of newly created solutions.

22

22

Genetic Algorithm: How Does It Work?

11. Form the Next Generation
    - Combine or replace individuals to create the next population.
    - Preserve the best solutions (elitism) when appropriate.
12. Repeat & Terminate
    - Repeat the evolutionary cycle until a stopping criterion is met.

Validate the Best Solution
- Test the final solution on unseen data or independent test cases, when applicable.

23

23

Genetic Algorithm: How Does It Work?

Evolutionary Cycle

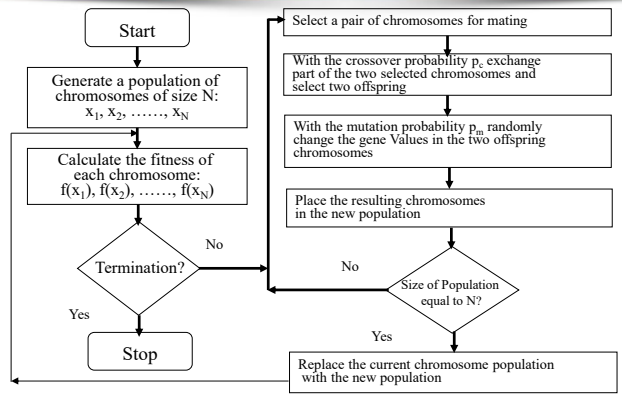
Initialize → Evaluate → Select → Crossover → Mutation → Repair/Penalize → Replace → Repeat

Validate

24

24

Genetic Algorithm Overview



25

25

Genetic Algorithm: Sample Problem

Problem: *Find the maximum value of the function ($15x-x^2$),*
where parameter x varies between 0 and 15.

Solution:

- Chromosome can be built with only four gene (0-15)
- *Suppose*:
  - The size of the chromosome population N is 6.
  - The crossover probability pc=0.7
  - The mutation probability=0.001
  - The fitness function $f(x)=15x-x^2$

26

26

Genetic Algorithm: Sample Problem

Step 1: GA creates an initial population of chromosomes by filling six 4-bit string with randomly generated ones and zeros

Chromosome Label	Chromosome String	Decoded Integer	Chromosome Fitness	Fitness Ratio
X1	1100	12		
X2	0100	4		
X3	0001	1		
X4	1110	14		
X5	0111	7		
X6	1001	9		

27

27

Genetic Algorithm: Sample Problem

Step 2: Calculate the fitness of each individual Chromosome

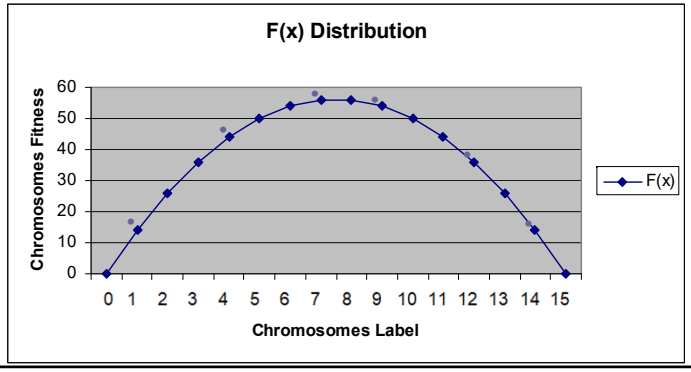
Chromosome Label	Chromosome String	Decoded Integer	Chromosome Fitness	Fitness Ratio
X1	1100	12	36	16.51
X2	0100	4	44	20.18
X3	0001	1	14	6.42
X4	1110	14	14	6.422
X5	0111	7	56	25.68
X6	1001	9	54	24.77

28

28

Genetic Algorithm: Sample Problem

Fitness Function and Chromosome Locations



29

29

Genetic Algorithm: Sample Problem

Step 3: The fitness ratio determines the chromosome's chance of being selected for mating. *X5 & X6 have fair chance*; *X3 & X4 have less chance*.

Chromosome Label	Chromosome String	Decoded Integer	Chromosome Fitness	Fitness Ratio
X1	1100	12	36	16.51
X2	0100	4	44	20.18
X3	0001	1	14	6.42
X4	1110	14	14	6.422
X5	0111	7	56	25.68
X6	1001	9	54	24.77

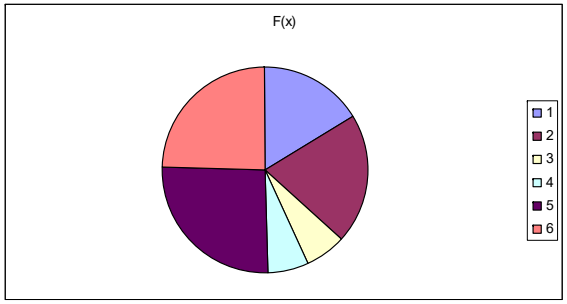
30

30

Genetic Algorithm: Sample Problem

Fitness Function in Roulette Wheel Distribution

1. 1100 (12)=36
2. 0100 (4)=44
3. 0001 (1)=14
4. 1110 (14)=14
5. 0111 (7)=56
6. 1001 (9)=54



31

31

Genetic Algorithm: Sample Problem

Step 4: Selected chromosome for mating. First X6 & X2 become parents.

Crossover Operator randomly chooses a crossover point where two parent chromosomes 'Break' and then exchange the chromosomes after that point.

32

Genetic Algorithm: Sample Problem

Step 5: Selected chromosome for mating. First X6 & X2 become parents

Generation i		Crossover
Chromosome Label	Chromosome String	
$X1_i$	1100	$X6_i$ 1 0 *0 1* → $X6_{i\_c}$ 10 *0 0*
$X2_i$	0100	$X2_i$ 0 1 *0 0* → $X2_{i\_c}$ 01 *0 1*
$X3_i$	0001	
$X4_i$	1110	
$X5_i$	0111	
$X6_i$	1001	

Genetic Algorithm: Sample Problem

Step 6: The mutation operator flips a randomly selected gene in a chromosome.

Generation i		Mutation
Chromosome Label	Chromosome String	
$X1_i$	1100	$X4_i$ 1 *0* 1 1 → $X4_{i\_c}$ 1 1 1 1
$X2_i$	0100	
$X3_i$	0001	
$X4_i$	1110	
$X5_i$	0111	
$X6_i$	1001	

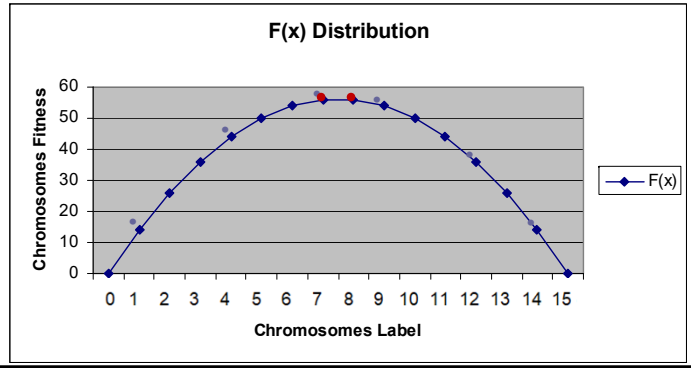
Genetic Algorithm: Sample Problem

Step 7: The fitness ratio determines the chromosome's chance of being selected for mating.

Generation i		Generation i+1		
Chromosome Label	Chromosome String	Chromosome Label	Chromosome String	Fitness
$X1_i$	1100	$X1_{i+1}$	1000	**56**
$X2_i$	0100	$X2_{i+1}$	0101	50
$X3_i$	0001	$X3_{i+1}$	1011	44
$X4_i$	1110	$X4_{i+1}$	0100	44
$X5_i$	0111	$X5_{i+1}$	0110	54
$X6_i$	1001	$X6_{i+1}$	0111	**56**

Genetic Algorithm: Sample Problem

Step 7: The fitness ratio determines the chromosome's chance of being selected for mating.



Genetic Algorithms: Evolutionary Operators

Selection, Crossover, and *Mutation*

Evolutionary operators determine how the population changes over time.

They control the trade-off between:

- *Exploration*: searching new regions
- *Exploitation*: refining promising regions

37

37

Genetic Algorithms: Evolutionary Operators

Selection is the process of choosing individuals from the current population to act as parents for the next generation.

Selection is usually biased by fitness.

A good selection method should:

- Give *better individuals* higher reproductive opportunity
- Avoid letting one individual *dominate* too early
- Preserve enough *diversity* for continued search
- Be computationally efficient

38

38

Genetic Algorithms: Evolutionary Operators

Selection pressure describes how strongly the algorithm favours high-fitness individuals.

Low pressure: weak bias toward better individuals

- Better diversity
- More exploration
- Slower improvement

High pressure: strong bias toward better individuals

- Faster convergence
- More exploitation
- Higher risk of premature convergence

39

39

Genetic Algorithms: Evolutionary Operators

Crossover recombines genetic material from two or more parents to create offspring.

Purpose:

- Combine useful partial solutions
- Exploit structure discovered in the population
- Produce new candidates without starting from scratch

Crossover is most effective when the representation places related genes in meaningful positions.

40

40

Genetic Algorithms: Evolutionary Operators

One-Point Crossover

Choose a crossover point and swap the tails.

```
Parent 1: 1 0 1 | 1 0 0
Parent 2: 0 1 0 | 0 1 1

Child 1:  1 0 1 | 0 1 1
Child 2:  0 1 0 | 1 0 0
```

Works well when nearby genes are related.

41

41

Genetic Algorithms: Evolutionary Operators

Two-Point Crossover

Choose two crossover points and swap the middle segment.

```
Parent 1: 1 0 | 1 1 0 | 0
Parent 2: 0 1 | 0 0 1 | 1

Child 1:  1 0 | 0 0 1 | 0
Child 2:  0 1 | 1 1 0 | 1
```

Useful when meaningful building blocks appear as segments.

42

42

Genetic Algorithms: Evolutionary Operators

Mutation randomly modifies one or more genes in a candidate solution.

Mutation supports:

- Exploration
- Diversity maintenance
- Escape from local optima
- Recovery of lost genetic material

Without mutation, the population can only recombine information already present.

43

43

Genetic Algorithms: Evolutionary Operators

Bit-Flip Mutation

For binary encoding:

```
Before 1 0 1 1 0 0 1
Mutate at position 3
After  1 0 0 1 0 0 1
```

Mutation probability is usually applied per gene.

44

44

Genetic Algorithms: Evolutionary Operators

Very low mutation:

- Population becomes similar
- Search may stagnate
- Lost genes may never return

Very high mutation:

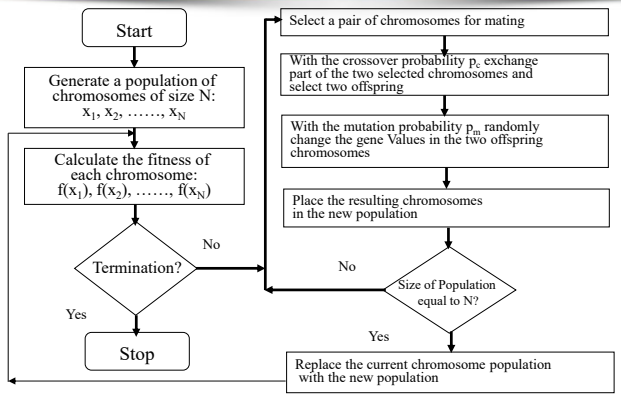
- Search becomes close to random
- Good building blocks are destroyed
- Convergence slows

Mutation must be strong enough to explore but not so strong that learning is erased.

45

45

Genetic Algorithm Overview



46

46

Genetic Algorithm: Common Mistakes

- Using training accuracy as fitness
- Ignoring computational cost
- Choosing an encoding that creates invalid offspring
- Reporting one run only
- Forgetting baseline comparisons
- Not monitoring diversity
- Treating GA as guaranteed global optimisation

47

47

Thank You & Questions

Abubakar.Siddique@aut.ac.nz

48

48